디지털 시스템 설계 — 수업정리노트

Ch0: Introduction & Trend in Computer Systems

과목 개요

이 과목 “Digital Systems Design”은 서울대 심재웅 교수님이 삼성DS System Expert 과정을 위해 진행하는 수업이다. 핵심 목표는 CPU/GPU/TPU와 같은 작은 하드웨어 조각을 직접 설계할 수 있는 능력을 기르는 것이다.

왜 이 과목이 필요한가? 소프트웨어 엔지니어가 하드웨어의 동작 원리를 이해하면, 시스템 레벨에서의 최적화 판단이 가능해진다. 특히 삼성DS처럼 반도체를 설계하는 조직에서는 SW/HW 인터페이스를 이해하는 것이 핵심 역량이다.

과목에서 다루는 주제는 시스템 스택의 Logic 레벨에 해당한다:

```
Algorithms
  └ Prog. Model/Lang.
       └ System Software
            └ Compiler
                 └ SW/HW Interface
                      └ Micro-architecture
                           └ Logic          ◀◀◀ 여기!
                                └ Devices
                                     └ Physics
```

구체적 토픽: 조합/순차 논리 복습 → Verilog HDL → 수 체계 & 부동소수점 → 하드웨어 빌딩 블록(Adder, Multiplier, SRAM, DRAM) → DNN 소개 → FPGA로 CNN 가속기 구현

컴퓨터 시스템의 트렌드

프로세서 진화의 역사

1971년 Intel 4004 (2,300 트랜지스터, 0.74MHz)에서 시작하여 오늘날 540억 개 이상의 트랜지스터를 가진 Manycore 프로세서까지, 2천만 배 이상 증가했다. 그 과정에서 성능을 끌어올린 핵심 기법들은 시대별로 달랐다:

```
1971: 4004 (0.74MHz)
  │  ← Clock Speed 향상
1978: 8086 (10MHz)
  │  ← Bus Width 확장
1993: Pentium (200MHz, Superscalar)
  │  ← Pipeline Width 확장
2006: Core2Duo (3.3GHz, Dual Core)
  │  ← Core 수 증가
TODAY: Manycore
```

Moore’s Law (무어의 법칙)

Intel 공동창업자 Gordon Moore가 제안한 경험 법칙으로, 칩 위의 트랜지스터 수가 약 12~24개월마다 2배로 증가한다는 것이다. 이 법칙은 50년간 놀랍도록 잘 맞아왔지만, 현재는 둔화되고 있다.

핵심 직관: 트랜지스터가 작아지면 → 같은 면적에 더 많이 넣을 수 있고 → 더 복잡한 회로를 만들 수 있다.

Dennard Scaling (데나드 스케일링)

Bob Dennard (1974)가 관찰한 법칙: 트랜지스터가 작아져도 전력 밀도(power density)는 일정하게 유지된다.

핵심 수식:

```
P ∝ C · V² · F

여기서:
  P = 전력 (Power)
  C = 커패시턴스 (Capacitance) — 트랜지스터 크기에 비례
  V = 전압 (Voltage)
  F = 주파수 (Frequency)
```

트랜지스터가 작아지면 C가 줄어들고, V도 낮출 수 있어서, F를 올려도 전력이 비슷했다. 하지만 2000년대 중반 이후 전압을 더 이상 낮출 수 없게 되면서 Dennard Scaling은 끝났다. 이것이 주파수 경쟁이 멈추고 멀티코어 시대로 전환된 이유다.

오늘날의 도전과 해법

성능 향상의 원천을 분석하면 (AMD Lisa Su 발표 기준): - 공정 기술: 40% - 마이크로아키텍처: 17% - 전력 관리: 15% - 다이 크기 증가: 12% - 추가 TDP: 8% - 컴파일러: 8%

현재의 도전: Moore's Law 둔화 + Dennard Scaling 종료 + 다이 크기 레티클 한계(~800mm²) + 수율 좋은 mm²당 비용 증가

해법은 특수 목적 가속기(Specialized Accelerators)이다:

- Google TPU v1: 28nm, 331mm², 700MHz, 92 TOPs (INT8), 40W
- Google TPU v2/v3: HBM 탑재, bfloat16 학습 지원, 최대 105 TFLOPs
- Tesla FSD Chip: Samsung 14nm, 260mm², 36W, NPU 2개 (각 36.86 INT8 TOPs)
- AWS Inferentia: FP16/BFP16/INT8 지원, 최대 128 TOPS
- Intel Loihi: 뉴로모픽 칩, SNN(Spiking Neural Network) 전용, 14nm 60mm²
- Google 양자컴퓨팅 등

핵심 메시지: "A New Golden Age for Computer Architecture" — 하드웨어 플랫폼과 애플리케이션이 계속 진화하고 있으며, 시스템 스택 전체를 재설계할 필요가 있다. 하지만 그 전에 디지털 시스템의 기초(fundamentals)를 이해해야 한다.

Ch1: Review — Logic Design

이 챕터는 Verilog를 배우기 전에 디지털 논리 설계의 기초를 빠르게 복습하는 것이 목적이다. 조합 논리(Combinational Logic)와 순차 논리(Sequential Logic)를 모두 다룬다.

1. Logic Gates (논리 게이트)

논리 게이트는 하나 이상의 이진 입력을 받아 하나의 이진 출력을 생성하는 디지털 회로의 가장 기본 요소이다. 각 게이트는 세 가지로 설명된다: 심벌(회로 기호), 불리언 수식, 진리표.

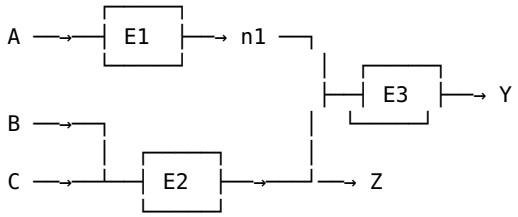
Gate	Boolean Eq.	Truth Table (A,B→Y)
BUF	$Y = A$	0→0, 1→1
NOT	$Y = \overline{A}$	0→1, 1→0
AND	$Y = A \cdot B$	00→0,01→0,10→0,11→1
OR	$Y = A+B$	00→0,01→1,10→1,11→1
XOR	$Y = A \oplus B$	00→0,01→1,10→1,11→0
NAND	$Y = \overline{(A \cdot B)}$	00→1,01→1,10→1,11→0
NOR	$Y = \overline{(A+B)}$	00→1,01→0,10→0,11→0

NOT 게이트의 출력 쪽에 있는 작은 원(bubble)은 반전(inversion)을 의미한다. NAND = AND + bubble, NOR = OR + bubble.

주의사항: XOR은 "둘 중 하나만 1일 때 1"이고, XNOR(= XOR + NOT)은 "둘 다 같을 때 1"이다. XOR은 패리티 검사, 가산기 등에서 핵심적으로 사용된다.

2. Digital Circuits (디지털 회로)

디지털 회로는 이산 값(discrete-valued) 변수를 처리하는 네트워크이다. 회로는 요소(elements)와 노드(nodes)로 구성된다.




```
입력 노드: A, B, C
출력 노드: Y, Z
내부 노드: n1
요소: E1, E2, E3
```

각 요소는 functional spec(입력→출력 관계)과 timing spec(지연 시간)으로 정의된다.

회로의 두 종류

조합 회로 (Combinational): 출력이 현재 입력의 조합에만 의존. 메모리 없음.

순차 회로 (Sequential): 출력이 입력의 시퀀스(현재 + 이전 값)에 의존. 메모리 있음.

이 구분이 중요한 이유: 조합 회로는 같은 입력이면 항상 같은 출력을 내지만, 순차 회로는 과거 이력에 따라 출력이 달라진다. 현실의 거의 모든 유용한 디지털 시스템은 두 가지를 조합해서 만든다.

3. Boolean Equations & Terminology (불리언 수식과 용어)

불리언 수식은 TRUE(1) 또는 FALSE(0)인 변수들의 식으로, 디지털 논리를 수학적으로 기술하는 데 완벽하다. 진리표가 주어지면 불리언 수식을 쓸 수 있다.

핵심 용어

Literal(리터럴): 변수 또는 그 보수. 예: A, $\bar{A}$, B, $\bar{B}$

Product(곱) = AND of literals. 예: A·B, $\bar{A}$·B·C Sum(합) = OR of literals. 예: A+B, $\bar{A}$+B+C

Implicant(함의항): 하나 이상의 리터럴의 AND. 예: A·B, $\bar{A}$·B·C, B 모두 3변수 함수의 implicant

Minterm(최소항): 함수의 모든 입력 변수를 포함하는 곱. 3변수 함수에서 $\bar{A}$·B·C는 minterm이지만, A·B는 C가 빠졌으므로 minterm이 아니다.

Maxterm(최대항): 함수의 모든 입력 변수를 포함하는 합. 3변수 함수에서 A+$\bar{B}$+C는 maxterm이지만, A+C는 B가 빠졌으므로 maxterm이 아니다.

Minterm/Maxterm 표 (3변수)

A B C	Minterm	Name	Maxterm	Name
0 0 0	$\bar{A}\cdot\bar{B}\cdot\bar{C}$	m0	A+B+C	M0
0 0 1	$\bar{A}\cdot\bar{B}\cdot C$	m1	A+B+$\bar{C}$	M1
0 1 0	$\bar{A}\cdot B\cdot\bar{C}$	m2	A+$\bar{B}$+C	M2
0 1 1	$\bar{A}\cdot B\cdot C$	m3	A+$\bar{B}$+$\bar{C}$	M3
1 0 0	$A\cdot\bar{B}\cdot\bar{C}$	m4	$\bar{A}$+B+C	M4
1 0 1	$A\cdot\bar{B}\cdot C$	m5	$\bar{A}$+B+$\bar{C}$	M5
1 1 0	$A\cdot B\cdot\bar{C}$	m6	$\bar{A}$+$\bar{B}$+C	M6
1 1 1	$A\cdot B\cdot C$	m7	$\bar{A}$+$\bar{B}$+$\bar{C}$	M7

규칙: minterm에서 변수가 0이면 보수($\bar{A}$), 1이면 true form(A). Maxterm은 반대: 0이면 true form(A), 1이면 보수($\bar{A}$).

왜 “min”term과 “max”term인가?

이름의 직관: - SOP(Sum of Products) = minterm들의 OR. OR의 최소 만족성: minterm 하나만 TRUE여도 출력이 TRUE → “min” - POS(Product of Sums) = maxterm들의 AND. AND의 최대 만족성: maxterm 전부가 TRUE여야 출력이 TRUE → “max”

SOP Canonical Form (정규 곱의 합)

진리표에서 출력 Y가 1인 행의 minterm들을 OR로 연결하면 된다.

예: 2변수 함수에서 Y가 AB=01, AB=11에서 1이면:

```
Y = Ā·B + A·B
Sigma 표기: F(A,B) = Σ(1, 3) 또는 Σ(m1, m3)
```

POS Canonical Form (정규 합의 곱)

진리표에서 출력 Y가 0인 행의 maxterm들을 AND로 연결한다.

예: 같은 함수에서 Y가 AB=00, AB=10에서 0이면:

```
Y = (A+B)·(Ā+B)
Pi 표기: F(A,B) = Π(0, 2) 또는 Π(M0, M2)
```

주의: SOP와 POS는 같은 함수를 다른 형태로 표현한 것이다. 어느 쪽이 더 간단한지는 함수에 따라 다르다.

4. Boolean Algebra (불리언 대수)

진리표에서 직접 쓴 불리언 수식은 가장 간단한 형태가 아닐 수 있다. 불리언 대수를 이용하면 수식을 간소화할 수 있고, 간단한 수식 = 더 적은 게이트 = 더 작고 빠르고 에너지 효율적인 회로가 된다.

Duality (쌍대성)

불리언 대수의 모든 공리와 정리는 쌍대 원리를 따른다: 0↔1과 ·(AND)↔+(OR)를 서로 바꾸면 여전히 성립하는 식이 된다.

Axioms (공리)

	Axiom		Dual	Name
A1	B=0 if B≠1	A1'	B=1 if B≠0	Binary
A2	$\overline{0} = 1$	A2'	$\overline{1} = 0$	NOT
A3	0·0 = 0	A3'	1+1 = 1	AND/OR
A4	1·1 = 1	A4'	0+0 = 0	AND/OR
A5	0·1 = 1·0 = 0	A5'	1+0 = 0+1 = 1	AND/OR

Theorems of One Variable (단일 변수 정리)

	Theorem		Dual	Name
T1	B·1 = B	T1'	B+0 = B	Identity
T2	B·0 = 0	T2'	B+1 = 1	Null Element
T3	B·B = B	T3'	B+B = B	Idempotency
T4	$\overline{\overline{B}} = B$			Involution
T5	$B \cdot \overline{B} = 0$	T5'	$B + \overline{B} = 1$	Complements

직관 설명: - Identity(T1): 1과 AND하면 자기 자신, 0과 OR하면 자기 자신 - Null(T2): 0과 AND하면 무조건 0, 1과 OR하면 무조건 1 - Idempotency(T3): 자기 자신끼리 AND/OR하면 자기 자신 - Involution(T4): 두 번 반전하면 원래대로 - Complements(T5): 자기 자신과 보수를 AND하면 0, OR하면 1

Theorems of Several Variables (다변수 정리)

	Theorem		Dual	Name
T6	B·C = C·B	T6'	B+C = C+B	Commutativity
T7	(B·C)·D = B·(C·D)	T7'	(B+C)+D = B+(C+D)	Associativity
T8	(B·C)+(B·D) = B·(C+D)	T8'	(B+C)·(B+D) = B+(C·D)	Distributivity
T9	B·(B+C) = B	T9'	B+(B·C) = B	Covering

T10	$(B\cdot C)+(B\cdot\bar{C}) = B$	T10'	$(B+C)\cdot(B+\bar{C}) = B$
Combining			
T11	$(B\cdot C)+(\bar{B}\cdot D)+(C\cdot D)$	T11'	$(B+C)\cdot(\bar{B}+D)\cdot(C+D)$
Consensus			
	$= B\cdot C + \bar{B}\cdot D$		$= (B+C)\cdot(\bar{B}+D)$
T12	$(B_0\cdot B_1\cdot B_2...)^{\overline{\ }}$	T12'	$(B_0+B_1+B_2...)^{\overline{\ }}$
De Morgan's			
	$= \bar{B}_0+\bar{B}_1+\bar{B}_2...$		$= \bar{B}_0\cdot\bar{B}_1\cdot\bar{B}_2...$

특히 중요한 정리들:

- T9 (Covering): $B\cdot(B+C) = B$. 직관: B가 이미 1이면 (B+C)도 반드시 1이므로 C는 무관.
- T10 (Combining): $B\cdot C + B\cdot\bar{C} = B$. 이것이 간소화의 핵심이다. C와 $\bar{C}$가 둘 다 있으면 C를 제거할 수 있다.
- T11 (Consensus): 세 번째 항(C·D)이 처음 두 항으로부터 파생되는 중복 항이므로 제거 가능.
- T12 (De Morgan's): AND의 반전 = 반전들의 OR, OR의 반전 = 반전들의 AND.

De Morgan's Theorem의 중요성

```
NAND ≡ OR with inverted inputs:    (A·B)¯= Ā + B̄
NOR  ≡ AND with inverted inputs:   (A+B)¯= Ā · B̄
```

이로부터 Logical Completeness(논리적 완전성): AND, OR, NOT만으로 모든 회로를 만들 수 있다. 더 나아가 NAND 게이트만으로(또는 NOR 게이트만으로) 모든 회로를 구현할 수 있다! 실제 반도체 제조에서 NAND/NOR 기반 설계가 많이 쓰이는 이유이다.

간소화 예제

$\bar{A}\cdot\bar{B}\cdot C + \bar{A}\cdot B\cdot C + A\cdot\bar{B}\cdot C$ 를 간소화:

```
단계 1: Ā·B̄·C + Ā·B·C                    (앞 두 항에서 B̄와 B로 T10 적용)
      = Ā·C·(B̄+B)  = Ā·C·1 = Ā·C

단계 2: Ā·C + A·B̄·C                      (C를 공통으로)
      = C·(Ā + A·B̄)
      = C·(Ā+A)·(Ā+B̄)                   (T8' 분배법칙)
      = C·1·(Ā+B̄) = C·(Ā+B̄)

결과: Y = Ā·C + B̄·C  또는 동치: Y = C·(Ā+B̄)
```

Prime Implicant(주요 함의항): Implicant가 다른 implicant와 더 이상 합쳐질 수 없을 때 prime implicant라고 한다. 최소화된 수식의 모든 항은 반드시 prime implicant여야 한다.

5. Karnaugh Maps (K-Map, 카르노 맵)

1953년 Bell Labs의 Maurice Karnaugh가 발명한 그래픽 기반의 불리언 수식 간소화 방법이다.

핵심 아이디어

합칠 수 있는 항들(즉, 하나의 변수만 다른 항들)을 그리드에서 인접하게 배치한다. 이를 위해 Gray Code 순서를 사용한다 (00, 01, 11, 10 — 연속된 값이 1비트만 차이남).

2변수 K-Map:

```
        A
      0   1
    ┌─────┬─────┐
B 0 │ m0  │ m2  │
    ├─────┼─────┤
  1 │ m1  │ m3  │
    └─────┴─────┘
```

3변수 K-Map:

```
          AB
       00  01  11  10
    ┌────┬────┬────┬────┐
C  0│ m0 │ m2 │ m6 │ m4 │
    ├────┼────┼────┼────┤
   1│ m1 │ m3 │ m7 │ m5 │
    └────┴────┴────┴────┘
```

K-Map 간소화 규칙

1. 모든 1을 포함하도록 가능한 한 적은 수의 사각형으로 묶는다
2. 각 사각형 안에는 1만 있어야 한다
3. 각 사각형은 2의 거듭제곱 크기의 직사각형이어야 한다 (1, 2, 4, 8, ...)
4. 각 사각형은 가능한 한 크게
5. K-Map의 가장자리는 연결된다 (좌우 wrap-around, 상하 wrap-around)
6. 하나의 1이 여러 사각형에 포함될 수 있다 (묶는 수를 줄이기 위해)

6. Combinational Building Blocks (조합 빌딩 블록)

Multiplexer (Mux, 멀티플렉서)

선택 신호(select signal)에 따라 여러 입력 중 하나를 출력으로 선택하는 회로.

```
2:1 Mux:

  D₀ ──┤ 0 │
       │   ├── Y
  D₁ ──┤ 1 │
       └───┘
         │
         S
```

수식: $Y = D_0 \cdot \overline{S} + D_1 \cdot S$

$S=0 \rightarrow Y=D_0$
$S=1 \rightarrow Y=D_1$

핵심 성질: 2^(N-1) 입력 Mux로 N 입력 논리 함수를 구현할 수 있다. Mux는 일종의 lookup table(LUT)로 동작한다. 이것이 FPGA의 기본 원리이기도 하다.

Decoder (디코더)

N개 입력, 2^N개 출력. 입력 조합에 따라 출력 하나만 활성화(assert).

디코더를 OR 게이트와 결합하면 모든 논리 함수를 구현할 수 있다. 각 출력이 하나의 minterm에 대응하기 때문이다.

X's and Z's

실제 회로에서는 0과 1 외에 두 가지 특수 값이 있다:

X (Illegal/Unknown value): 회로 노드의 값을 알 수 없는 상태. 시뮬레이터에서 초기화되지 않은 값을 나타낼 때도 사용.

Z (Floating/High-impedance value): 노드가 HIGH도 LOW도 아닌 상태. tri-state 버퍼의 출력이 비활성화될 때 나타남.

주의: X와 Z는 Verilog에서 매우 중요하게 사용된다. X는 "don't care"로 K-Map 최적화에서도 활용될 수 있다.

7. Sequential Circuits (순차 회로)

순차 논리는 메모리를 가진다. 이전의 입력을 명시적으로 기억하거나, 이전 입력을 시스템의 상태(state)로 축약한다.

Bistable Element (쌍안정 소자)

메모리의 가장 기본 빌딩 블록. 두 개의 안정 상태를 가진 소자이다.

인버터 두 개를 루프로 연결하면 간단한 bistable element를 만들 수 있다:

```
Q──┤  I1  ├──→──┤  I2  ├──→──Q
   └──────┘     └──────┘
        ↑                     │
        └─────────────────────┘
```

- Q=0이면: I2 입력=0 → I2 출력=1 → I1 입력=1 → I1 출력=0 → Q=0 (안정!)
- Q=1이면: I2 입력=1 → I2 출력=0 → I1 입력=0 → I1 출력=1 → Q=1 (안정!)

N개의 안정 상태를 가진 소자는 $\log_2 N$ 비트의 정보를 저장한다.

문제점: 상태를 바꿀 수 있는 입력이 없다! → Latch와 Flip-Flop이 필요.

8. Latches and Flip-Flops

Latch vs Flip-Flop의 핵심 차이

Latch는 신호 레벨에 반응(CLK=1일 때 transparent)하고, Flip-Flop은 클럭 에지(전이)에 반응(에지 순간에만 값 갱신)한다. Latch는 Level-sensitive, Flip-Flop은 Edge-triggered라고도 부른다.

SR (Set-Reset) Latch

두 개의 교차 결합된 NOR (또는 NAND) 게이트로 구성.

```
R ──→──┤ N1 ├──o──→ Q
       │    │     \
       └────┘      \
            \       \
             \       |
```

```
                    │
S ──→─┤ N2 ├──o──→ Q̄
```

진리표:

Case	S	R	Q	$\bar{Q}$	
IV	0	0	Qprev	$\bar{Q}$prev	(유지 — 메모리!)
I	0	1	0	1	(Reset)
II	1	0	1	0	(Set)
III	1	1	0	0	(금지! Q와 $\bar{Q}$ 모순)

함정: S=R=1은 Q=$\bar{Q}$=0이 되어 "Q와 $\bar{Q}$이 서로 보수"라는 전제가 깨진다. 이 상태에서 S와 R을 동시에 0으로 놓으면 출력이 불확정(race condition)이 된다.

D Latch

SR Latch의 두 가지 문제를 해결: 1. S=R=1 상태가 발생하지 않도록 → 데이터 입력 D 하나로 S와 R을 자동 생성 (S=D, R=$\bar{D}$) 2. 무엇을(D) 바꿀지와 언제(CLK) 바꿀지를 분리

```
CLK=1: transparent (Q follows D)
CLK=0: opaque (Q holds previous value)
```

주의사항: D Latch는 CLK=1인 동안 계속 D를 통과시킨다(transparent). 이것이 문제가 되는 이유는 뒤에서 설명.

D Flip-Flop

두 개의 D Latch를 back-to-back으로 연결 (master + slave) + 상보적 클럭.

```
                  CLK
                   │
                ┌──○──┐
                │     │
       CLK      │     │   CLK
D ──→─┤ D  Q ├──→N1──→┤ D  Q ├──→ Q
      │ L1 Q̄├      │ L2 Q̄├──→ Q̄
       └──────┘      └──────┘
       master        slave
```

동작: - CLK=0: master transparent (D → N1), slave opaque (N1 차단) - CLK=1: master opaque (D 차단), slave transparent (N1 → Q) - 결과: rising edge (0→1)에서만 D의 값이 Q로 복사되고, 나머지 시간에는 값 유지

이것이 디지털 회로에서 가장 핵심적인 저장 소자인 이유: 클럭 에지에서만 값이 바뀌므로 timing을 정확하게 제어할 수 있다.

Register (레지스터)

공통 CLK을 공유하는 N개 flip-flop의 뱅크. 대부분 순차 회로의 핵심 빌딩 블록이다.

9. Synchronous vs Asynchronous Sequential Circuits

동기식(Synchronous): 상태 전이가 클럭 에지에서만 발생. 상태가 클럭에 동기화됨. 거의 모든 실용적 디지털 시스템이 이 방식.

비동기식(Asynchronous): 클럭 없이 입력 변화에 즉시 반응. 설계가 매우 어렵다. 예: Ring Oscillator (홀수 개의 인버터를 루프로 연결 → 출력이 계속 진동).

동기식을 선호하는 이유: 클럭이 있으면 모든 상태 변화의 타이밍을 예측할 수 있어서, 설계·검증·디버깅이 훨씬 용이하다.

10. Finite State Machines (FSMs, 유한 상태 기계)

FSM은 k개의 레지스터를 가진 동기식 순차 회로로, 최대 2^k개의 고유 상태 중 하나에 있을 수 있다.

FSM의 구성

```
FSM = Next State Logic + State Register + Output Logic
```

Moore vs Mealy Machine

```
Moore Machine:
                          CLK
                           |
  inputs ─→┤ Next State ├─→┤ Register ├─→┤ Output Logic ├─→ outputs
      M     │   Logic    │  k│          │k │              │  N
            └────────────┘   └──────────┘  └──────────────┘

  출력 = f(현재 상태만)


Mealy Machine:
                          CLK
                           |
  inputs ─→┤ Next State ├─→┤ Register ├─→┤ Output Logic ├─→ outputs
      M     │   Logic    │  k│          │k │              │  N
            └────────────┘   └──────────┘  └──────────────┘
                                                ↑
                                            inputs (직접 연결)

  출력 = f(현재 상태 + 현재 입력)
```

Moore: 출력이 상태에만 의존 → 출력이 안정적, 한 클럭 늦게 반응 Mealy: 출력이 상태+입력에 의존 → 더 빠르게 반응 가능하지만, 입력 glitch에 민감

FSM 설계 예제: Traffic Light Controller

문제 설정: 두 교통 센서(T_A, T_B), 두 신호등(L_A, L_B: green/yellow/red), 5초 주기 클럭, Reset 버튼.

State Transition Diagram:

```
          T_A
     ┌──────────┐
     │          │
     ▼    T̄_A   │
  ┌──────┐ ──→ ┌──────┐
  │  S0  │     │  S1  │
  │LA:grn│     │LA:yel│
  │LB:red│     │LB:red│
  └──────┘     └──────┘
     ▲             │
     │   always    │
```

```
Reset│              ▼
     │            ┌─────┐
  ┌──────┐        │ S2  │
  │ S3   │ ←──    │LA:red│
  │LA:red│ T̄_B    │LB:grn│
  │LB:yel│        │     │
  └──────┘        └─────┘
     │              ▲
     │    always    │
     └──────T_B─────┘
```

State Encoding: S0=00, S1=01, S2=10, S3=11 Output Encoding: green=00, yellow=01, red=10

Next State Equations (간소화 후):

$$S'_1 = S_1 \oplus S_0$$
$$S'_0 = \bar{S}_1 \cdot \bar{S}_0 \cdot \bar{T}_A + S_1 \cdot \bar{S}_0 \cdot \bar{T}_B$$

Output Equations:

$$L_A1 = S_1, \quad L_A0 = \bar{S}_1 \cdot S_0$$
$$L_B1 = \bar{S}_1, \quad L_B0 = S_1 \cdot S_0$$

이것은 Moore machine이다. 출력(L_A1, L_A0, L_B1, L_B0)이 현재 상태(S_1, S_0)에만 의존하고, 입력(T_A, T_B)에 직접 의존하지 않기 때문이다.

Ch2: Introduction to Verilog

이 챕터는 Verilog HDL의 기초를 다룬다. 여기서 가장 중요한 세 가지를 먼저 짚으면: (1) wire vs reg의 차이, (2) Continuous Assignment vs Procedural Assignment의 차이, (3) 세 가지 모델링 레벨 (Structural/Dataflow/Behavioral)의 용도와 차이이다.

1. Hardware Description Language (HDL)이란?

하드웨어를 설계할 때 회로도(schematic)를 직접 그리는 방법이 있지만, 수백만~수십억 개의 트랜지스터를 가진 현대 칩에서는 불가능하다. 텍스트 기반으로 하드웨어를 기술할 수 있는 언어가 필요한데, 그것이 HDL이다.

Verilog: 1984년 Gateway Design Automation에서 개발, 1990년 Cadence에 인수. 1995년 IEEE 1364 표준이 됨. 버전: Verilog-95 → Verilog-2001 → Verilog-2005 → SystemVerilog (업계 표준).

VHDL (Very High Speed Integrated Circuit HDL): 1983년 미국 국방부에서 개발. 1987년 IEEE 1076 표준.

이 과목에서는 Verilog를 사용한다.

SW Programming vs HW Description — 핵심 차이

이것이 Verilog를 배울 때 가장 중요한 직관이다:

소프트웨어: 주어진 하드웨어 위에서 명령문을 순서대로 실행 HDL: 주어진 회로를 텍스트로 기술 — 모든 것이 본질적으로 동시에(concurrently) 동작

```
// 이것은 "a를 x에 복사하라"가 아니라
// "a에서 x로 가는 와이어 연결이 있다"는 의미!
assign x = a;
```

```
assign y = b;
// 두 assign문은 동시에 동작한다 (순서 무관)
```

assign은 RHS에서 LHS로 데이터가 흐르는 연결(connection)을 기술하는 것이지, RHS를 LHS로 복사하는 동작이 아니다.

Synthesizable Verilog

합성(Synthesis): HDL 코드 → Logic Gates & Wires (netlist) 변환.

Verilog 코드를 하드웨어로 매핑하려면 합성 가능(synthesizable)해야 한다. 모든 Verilog 구문이 합성 가능한 것은 아니다:

```
합성 가능:  module, instantiation, always, assign, wire, reg ...
합성 불가:  initial, $display, $monitor, #delay, $finish ...
            (이것들은 시뮬레이션/테스트벤치 전용)
```

주의: initial 블록은 합성 불가능하다! 테스트벤치에서만 사용. always 블록은 합성 가능하다(reg 타입 할당 시).

2. Module — Verilog의 기본 단위

Module은 Verilog에서 가장 핵심적인 빌딩 블록이다. 입력과 출력을 가진 모든 하드웨어 블록은 모듈이다 (AND 게이트, Mux, 디코더, CPU 전체 등).

모듈은 두 부분으로 구성된다: - Core circuit (internal/body): 필요한 기능을 수행하는 내부 회로 - Ports (interface): 내부 회로와 외부 세계를 연결하는 인터페이스

```
  ┌──────────────────────────┐
  │          Module          │
  │                          │
  │  ┌──────────────────┐    │
a─┤→│   Core Circuit    │    │
  │  │   (Internal/Body) │→──┤─→ y
b─┤→│                   │    │
  │  └──────────────────┘    │
  │        Ports (Interface) │
  └──────────────────────────┘
```

```
module test (input  a,
             input  b,
             output y);
  // describe our circuit here!
endmodule
```

주의: module 선언 끝의 세미콜론(;)을 잊지 말 것! endmodule 뒤에는 세미콜론이 없다.

ANSI-C 스타일 포트 선언을 사용하라. 읽기 쉽고 타이핑 실수가 적다.

3. Lexical Conventions (어휘 규칙)

Verilog의 어휘 규칙은 C 언어와 매우 유사하다.

식별자(Identifier): 영숫자, *, $로 구성. 대소문자 구분(case-sensitive). 첫 글자는 문자,* , 또는 $ (숫자로 시작 불가). 예: counter, four_bit_adder, _4b_adder

키워드(Keywords): 예약어. and, assign, begin, case, else, end, module, wire, reg, always 등. 모두 소문자.

주석(Comments): // 한 줄 주석, /* */ 여러 줄 주석.

숫자 표현 — 매우 중요!

형식: `<size>'<base>value`

- size: 비트 수. 생략 시 32비트로 가정
- ': 작은따옴표 (backtick ` 아님!)
- base: b/B (2진), o/O (8진), d/D (10진), h/H (16진). 생략 시 10진
- value: 상수 값

크기 불일치 시 처리 규칙:

```
크기 < 값:  MSB가 잘린다 (truncation)
            예: 4'b11110010 → 4'b0010 (상위 4비트 잘림)

크기 > 값:  왼쪽을 채운다 (padding)
            - 최상위가 0 또는 1이면 → 0으로 채움
            - 최상위가 x이면 → x로 채움
            - 최상위가 z이면 → z로 채움
```

숫자 표현 예제:

표현식	저장되는 값	
1	00000000000000000000000000000001	(32비트 decimal)
8'hAA	10101010	(8비트)
6'b10_0011	100011	(6비트)
'hF	00000000000000000000000000001111	(32비트)
6'hCA	001010	(6비트, CA=11001010 → 하위 6비트만)
6'hA	001010	(6비트, A=1010 → 0 패딩)
16'bZ	zzzzzzzzzzzzzzzz	(16비트 high-Z)
8'bx	xxxxxxxx	(8비트 unknown)

음수 표현: -'value. 2의 보수(2's complement) 형식.

```
a = 14'h1234       → 00001234
a = -14'h1234      → ffffedcc  (32비트 reg에서 2의 보수)
a = 32'hDEAD_BEEF  → deadbeef
a = -32'hDEAD_BEEF → 21524111  (2의 보수)
```

특수 문자: _(underscore)는 가독성 용도. ?(물음표)는 z와 동일. Four-value logic: 0, 1, x(unknown), z(high-impedance).

4. Data Types — wire vs reg

이것이 Verilog에서 가장 흔한 혼동 포인트이다.

Nets (연결선): wire, tri, wand, trior, wor 등. 하드웨어 연결점(connection point). 값을 저장할 수 없다! primitive, continuous assignment, module port에 의해 구동됨. Verilog의 wire는 방향성이 있다(directional).

Variables (변수): reg, integer, real, time, realtime. 데이터 저장 소자를 표현. procedural statement, task, function 내에서만 값 할당 가능. input이나 inout 포트가 될 수 없다.

함정 주의: reg가 반드시 물리적 레지스터(flip-flop)를 의미하는 것이 아니다! reg는 단지 “procedural block에서 값을 할당받을 수 있는 변수”라는 의미이다. 합성 시 조합 논리로 구현될 수도 있고, 레지스터로 구현될 수도 있다.

wire와 reg의 사용 규칙:

```
wire: continuous assignment (assign)의 좌변(LHS), module의
input/inout port
reg:  procedural block (always/initial)의 좌변(LHS), output port로 사
용 가능 (output reg)
```

5. Verilog Assignments — 세 가지 할당 방식

1. Continuous Assignment (연속 할당): net(wire)에 값을 구동(drive). 조합 논리를 모델링. assign 키워드 사용. 항상 구동 상태이다. RHS의 값이 바뀔 때마다 LHS가 즉시 업데이트된다.

```
wire temp;
assign temp = a | b;        // 명시적
wire temp = a | b;          // 암시적 — 선언과 할당 결합
```

2. Procedural Assignment (절차적 할당): reg 타입 변수에 always 블록/task/function 내에서 값 할당. 보통 레지스터와 FSM을 모델링.

3. Procedural Continuous Assignment (절차적 연속 할당): assign/deassign으로 변수에 대한 모든 절차적 할당을 override. 특수한 경우에 사용.

6. Verilog Ports (포트)

포트는 모듈을 외부 세계와 연결하는 창구이다.

```
  ┌──────────────────────────────────────┐
  │              Module                  │
  │                                      │
net/var ─→■ input port ─→ net            │
  │           net ←─ output port ■←─ net/var
  │                                      │
net ─→■←─ inout port ─→←─ net            │
  └──────────────────────────────────────┘
```

기본값: 포트는 기본적으로 wire 타입, unsigned. output 포트만 reg 타입으로 선언 가능(output reg).

Port Connection 두 가지 방식:

```
// 순서에 의한 연결 (positional) — 비추천
half_adder ha_1 (x, y, s1, c1);

// 이름에 의한 연결 (named) — 추천!
half_adder ha_2 (.x(cin), .y(s1), .s(s), .c(c2));
```

이름에 의한 연결을 사용하라! 순서가 무관하고, 포트가 많을 때 실수를 줄일 수 있다.

Unconnected/Floating ports: input이 연결되지 않으면 z로 구동, output이 연결되지 않으면 floating.

주의: built-in primitive 게이트(and, or, xor 등)는 instance name을 생략할 수 있지만, 사용자 정의 모듈을 인스턴스화할 때는 instance name이 필수이다.

7. Port Connection 전체 예제

```
module half_adder (input  x,
                   input  y,
                   output s,
                   output c);
  xor (s, x, y);    // Verilog built-in primitive
  and (c, x, y);    // instance name 생략 가능 (primitive만)
endmodule

module full_adder (input  x,
                   input  y,
                   input  cin,
                   output s,
                   output cout);
  wire s1, c1, c2;  // 내부 노드 선언

  half_adder ha_1 (x, y, s1, c1);                      //
positional
  half_adder ha_2 (.x(cin), .y(s1), .s(s), .c(c2));     // named
  or (cout, c1, c2);                                    //
primitive
endmodule
```

```
Full Adder 내부 구조:

x ──→┤ Half  ├─→ s1 ──→┤ Half  ├─→ sum
y ──→┤ Adder ├─→ c1    │ Adder ├─→ c2
     │ ha_1  │    │ cin──→┤ ha_2  │    │
     └───────┘    │       └───────┘    │
                  └──→┤ OR ├──→ cout ←┘
```

8. Module Modeling Levels — 세 가지 모델링 레벨

(1) Structural Modeling (구조적 모델링)

게이트 레벨에서 회로를 기술. Built-in primitive, 사용자 정의 primitive, 또는 다른 모듈을 wire로 연결. 계층 구조(hierarchy)를 기술.

```
// Half Adder: S = X⊕Y, C = XY
module half_adder(input x, input y, output s, output c);
  xor (s, x, y);
  and (c, x, y);
endmodule
```

(2) Dataflow Modeling (데이터플로우 모델링)

입력에서 출력으로의 데이터 흐름을 기술. 연산자와 continuous assignment(assign) 사용. 조합 논리에 적합.

```
module full_adder_dataflow(input x, input y, input c_in,
                           output sum, output c_out);
  assign #5 {c_out, sum} = x + y + c_in;
endmodule
```

{c_out, sum}은 concatenation 연산으로, c_out과 sum을 하나의 2비트 결과로 묶은 것이다.

(3) Behavioral Modeling (행위적 모델링)

always 블록과 procedural statement 사용. 대상(LHS)은 반드시 reg 타입. 순차 논리와 조합 논리 모두 가능.

```
module full_adder_behavioral(input x, input y, input c_in,
                             output reg sum, output reg c_out);
  always @(x, y, c_in)
    #5 {c_out, sum} = x + y + c_in;
endmodule
```

always @(x, y, c_in): sensitivity list에 있는 신호 중 하나라도 값이 바뀌면 always 블록이 활성화된다. always @(*)로 쓰면 블록 내에서 읽히는 모든 신호에 자동으로 민감하게 된다(Verilog-2001 이후, 조합 논리에서 추천).

(4) Mixed-Style Modeling

세 가지 모델링 레벨을 하나의 모듈 안에서 섞어 사용. 대규모 설계에서 흔히 사용.

```
module full_adder_mixed_style(input x, input y, input c_in,
                              output sum, output reg c_out);
  wire s1, c1, c2;

  // Structural: Half Adder 1
  xor xor_ha1(s1, x, y);
  and and_ha1(c1, x, y);

  // Dataflow: Half Adder 2
  assign sum = c_in ^ s1;
  assign c2  = c_in & s1;

  // Behavioral: Output OR gate
  always @(c1, c2)
    c_out = c1 | c2;
endmodule
```

세 가지가 모두 동시에(concurrently) 동작한다.

RTL (Register-Transfer Level) = synthesizable behavioral + dataflow. 업계에서 가장 많이 사용하는 추상화 수준이다.

9. Verilog Simulation

설계한 모듈을 검증하기 위해 별도의 테스트벤치(testbench)를 사용한다.

```
┌──────────────────────────────────────┐
│          Top-Level Block             │
│  ┌──────────────┐   ┌──────────────┐ │
│  │  Stimulus    │──→│ Module Under │ │
│  │  Block       │←──│ Test (MUT)   │ │
│  │ (Testbench)  │   └──────────────┘ │
│  └──────────────┘                    │
└──────────────────────────────────────┘
```

System Tasks:

```
$display(ep1, ep2, ..., epn);  // 값 출력 (한 번)
$monitor(ep1, ep2, ..., epn);  // 값 변화 시마다 자동 출력
$stop;                          // 시뮬레이션 일시 정지
$finish;                        // 시뮬레이션 종료
```

$display와 $monitor의 차이: $display는 호출 시점에 한 번만 출력하지만, $monitor는 인자의 값이 변할 때마다 자동으로 출력한다.

Timescale: `timescale time_unit / time_precision. 예:timescale 1ns / 1ps → #15는 15ns. FPGA 설계에서는 ns를 time unit으로 권장.

테스트벤치 예제 — Clock Generation:

```
module tb;
  reg clk;

  always #5 clk = ~clk;    // 5ns마다 반전 → 주기 10ns 클럭

  initial begin
    $monitor("Time = %0t clk = %0d", $time, clk);
    #5  clk = 0;    // 5ns에 clk을 0으로
    #80 $finish;    // 85ns에 시뮬레이션 종료
  end
endmodule
```

4:2 Encoder 테스트벤치 예제:

```
`timescale 1ns / 100ps

module encoder4to2_tb;
  reg  [3:0] din;
  wire [1:0] dout;

  encoder4to2 encoder(
    .Y0(din[0]), .Y1(din[1]),
    .Y2(din[2]), .Y3(din[3]),
    .A(dout)
  );

  initial begin
    $monitor("Time = %0t din = %0b dout = %0b", $realtime, din,
dout);
    #5    din = 4'b1000;   // 5ns
    #10   din = 4'b0100;   // 15ns
    #20   din = 4'b0010;   // 35ns
    #15   din = 4'b0001;   // 50ns
    #1    din = 4'b1000;   // 51ns
    #0.5  din = 4'b0100;   // 51.5ns
    #0.15 din = 4'b0010;   // 51.65ns → 51.7ns (100ps 해상도로 반올림)
    #10   $finish;          // 61.7ns
  end
endmodule
```

시뮬레이션 결과:

```
Time = 0       din = xxxx  dout = xx      (초기: 모든 값 unknown)
Time = 5000    din = 1000  dout = 11      (Y3=1 → A=3)
Time = 15000   din = 100   dout = 10      (Y2=1 → A=2)
Time = 35000   din = 10    dout = 1       (Y1=1 → A=1)
Time = 50000   din = 1     dout = 0       (Y0=1 → A=0)
Time = 51000   din = 1000  dout = 11
Time = 51500   din = 100   dout = 10
Time = 51700   din = 10    dout = 1       (51.65 → 51.7로 반올림)
```

주의: #0.15로 인한 51.65ns는 100ps 정밀도(timescale 1ns/100ps)에 의해 51.7ns로 반올림된다.

Ch3: Dataflow Modeling

이 챕터에서는 Verilog의 데이터플로우 모델링을 깊이 다룬다. 핵심은 두 가지: (1) Continuous Assignment의 상세 규칙과 지연 모델, (2) Verilog의 모든 연산자 종류와 동작 특성(특히 signed/unsigned 비교, x/z 처리, shift 연산).

1. Structural Modeling의 한계와 Dataflow Modeling의 동기

챕터 2에서 배운 Structural modeling(게이트 인스턴스화)은 복잡한 회로에서는 비현실적이다. 수백 개의 게이트를 일일이 연결하는 것은 유지보수가 불가능하다. Dataflow modeling은 게이트 프리미티브 대신 연산자를 사용하여 입력에서 출력으로의 데이터 흐름을 기술한다.

```
// Structural (불편)              // Dataflow (간결)
xor (s, x, y);                    assign sum   = x ^ y;
and (c, x, y);                    assign c_out = x & y;
```

2. Continuous Assignments — 상세 규칙

Continuous assignment는 dataflow modeling의 가장 기본 문장이다.

문법: assign [drive_strength] [delay] = ;

핵심 특성: - net(wire)에만 값을 구동 (variable/reg에는 사용 불가!) - vector와 scalar 모두 가능 - Always Active: RHS의 값이 변할 때마다 즉시 LHS가 갱신됨 - 조합 논리를 논리 표현식으로 모델링

선언과 할당 방식 3가지:

```
// (1) Regular: 별도 선언 + assign (추천)
wire out;
assign out = in1 & in2;

// (2) Implicit continuous assignment: 선언과 동시에 할당
wire out = in1 & in2;

// (3) Implicit net declaration: wire 선언 없이 assign (비추천!)
assign out = in1 & in2;   // out이 wire로 암시 선언됨
```

중요 규칙: - net은 한 번만 선언 가능. wire out = in1 & in2; 뒤에 wire out = in3;은 불법 - 하나의 net에 여러 continuous assignment 가능 (하지만 특별한 이유 없으면 하지 마라) - Implicit net declaration은 비추천: 원했던 게 reg인데 실수로 wire가 되는 버그 유발 - LHS는 반드시 scalar/vector net 또는 net의 concatenation (절대 reg가 아님!) - RHS에는 scalar/vector reg, function call 등 사용 가능

3. Delays (지연)

지연은 RHS 값이 변한 시점부터 LHS에 할당되기까지의 시간이다.

세 가지 지연 지정 방법:

```
// (1) Regular continuous assignment delay
assign #10 out = in1 & in2;   // inertial delay

// (2) Implicit continuous assignment delay
wire #10 out = in1 & in2;     // inertial delay
```

```
// (3) Net delay
wire #10 out;                    // transport delay
assign out = in1 & in2;
```

Inertial delay (관성 지연): 게이트의 RC를 모델링. continuous assignment에서 기본 사용. 지연보다 짧은 펄스는 무시(걸러짐).

Transport delay (전파 지연): 와이어의 지연을 모델링. net delay에서 기본 사용. 모든 변화가 그대로 전파됨.

합성 시 #delay는 무시된다. 시뮬레이션 전용.

4. Constants — signed/unsigned 상세 (시험 핵심 함정)

단순 10진수(-123, 12345)는 signed로 취급된다. Base format 표기(16'habcd)는 기본 unsigned이며, 's'를 쓰면 signed가 된다.

핵심 원칙 두 가지: - 원칙 1: 음수는 2의 보수로 표현된다. - 원칙 2: 's' 지정자는 비트 패턴을 바꾸지 않는다. 해석(interpretation)만 바꾼다.

상세 예제 (시험 필출):

```
4'sb1001  → 비트: 1001  → signed 해석 → -7
5'sb1001  → 비트: 01001 → signed 해석 → +9 (크기 큼 → 0 패딩)
-4'sb1001 → -(-7) = +7
4'shf     → 비트: 1111  → signed 해석 → -1
4'hf      → 비트: 1111  → unsigned 해석 → 15
5'shf     → 비트: 01111 → signed 해석 → +15
4'sd12    → 비트: 1100  → signed 해석 → -4  (12=1100)
5'sd12    → 비트: 01100 → signed 해석 → +12
3'sd12    → 비트: 100   → signed 해석 → -4  (truncation)
-4'sd12   → -(-4) = +4
-4'sb0010 → -(2) → 비트패턴: 1110
4'd -12   → 불법! (마이너스는 맨 앞에만)
```

5. Variable Data Types — 상세

reg: 할당 사이에 값을 유지. 기본 unsigned. signed 선언 가능. 초기값: x.

```
reg a, b;                    // 1-bit reg
reg [7:0] data_a;            // 8-bit, MSB = bit 7
reg signed [7:0] d;          // 8-bit signed reg
```

integer: signed reg로 취급, LSB at bit 0. 최소 32비트. 초기값: x.

time: unsigned, 최소 64비트. $time과 함께 시뮬레이션 시간 저장용. 초기값: x.

real / realtime: 서로 동일, 교환 가능. range 선언 불가. 초기값: 0.0 (reg/integer/time과 다름!).

6. Vectors, Bit-Select, Part-Select

Bit endianness가 중요하다:

```
reg [7:0] a = 14;    // MSB=bit7, a=00001110, a[3:0]=1110 ✓
reg [0:7] b = 14;    // MSB=bit0, b=00001110, b[0:3]=0000 ✓
                     // b[3:0]은 불법! (endianness 뒤집기 불가)
```

Variable part-select는 루프에서 벡터를 슬라이싱할 때 핵심적으로 사용된다:

```
wire [15:0] big_vect;
big_vect[8+:8]   // = big_vect[15:8]  (8에서 위로 8비트)
big_vect[7-:8]   // = big_vect[7:0]   (7에서 아래로 8비트)
// width는 반드시 상수, base는 변수 가능
```

7. Array and Memory

한 번에 하나의 배열 요소만 접근(할당) 가능:

```
reg [7:0] mema [7:0];           // 8개의 8-bit word
reg [7:0] memb [3:0][3:0];     // 4x4의 8-bit word

mema[4][3]          // 합법: 4번째 word의 3번 비트
mema[5][7:4]        // 합법: 5번째 word의 상위 4비트
memb[3][1][1:0]     // 합법: [3][1] word의 하위 2비트

mema = 0;           // 불법! 전체 메모리 한 번에 할당 불가
memb[1] = 0;        // 불법! 행 전체 할당 불가
memb[1][3:1] = 0;   // 불법! 여러 열 동시 할당 불가
mema[1] = 0;        // 합법 (단일 word)
memb[1][3] = 0;     // 합법 (단일 word)
```

8. Operators — Verilog 연산자 총정리

Arithmetic Operators (산술 연산자)

+, -, *, /, **, %. 피연산자에 x 또는 z 비트가 있으면 결과는 전부 x. 나눗셈/나머지에서 2번째 피연산자가 0이면 결과는 x. %(modulus)는 합성 가능하지만 가능하면 피하라(하드웨어 비용 높음).

Relational Operators (관계 연산자)

<, >, <=, >=. 결과: true→1, false→0, x/z 있으면→x.

signed/unsigned 비교 규칙 — 매우 중요한 함정!

규칙: 피연산자 중 하나라도 unsigned이면, 둘 다 unsigned로 비교. 둘 다 signed이면 sign-extension 후 signed 비교.

```
wire signed [3:0] a = -4'd4;    // a = 1100 (signed → -4)
wire unsigned [3:0] b = 4'd5;   // b = 0101 (unsigned → 5)

a > b → ?
// b가 unsigned → 둘 다 unsigned로 비교
// a = 1100 = 12(unsigned), b = 0101 = 5
// 12 > 5 → 결과: 1 (true!)
// signed로 보면 -4 > 5는 false인데, unsigned 비교라서 true!
```

크기가 다른 피연산자의 비교: 둘 다 signed → sign-bit extension + signed 비교. 한쪽이 unsigned → zero-padding + unsigned 비교.

Equality Operators (등호 연산자)

== (logical equality): x/z 비트가 있으면 결과가 x. === (case equality): x와 z까지 정확히 비교. 결과는 항상 0 또는 1. 합성 불가.

```
4'b1x0z === 4'b1x0z   // → 1 (정확히 일치)
4'b1x0z == 4'b1x0z    // → x (x/z 비트 있으므로)
```

Equality with sign extension:

```
wire signed [5:0] a = -6'd4;   // a = 111100
wire signed [3:0] b = -4'd4;   // b = 1100 → sign ext → 111100
a == b → 1 (true)
```

Concatenation & Replication Operators

{a, b[0], c[2:1]} → 비트를 이어붙임. 피연산자는 반드시 sized. {n{expr}} → 반복 concatenation.

```
a = 1'b1, b = 2'b00, c = 3'b101
{a, b[0], c[2:1]}  → 4'b1010
{ {1{a}}, {2{b[0]}}, {3{c[2:1]}} }  → 9'b1_00_10_10_10
```

응용 예제: 4-bit Adder

```
module four_bit_adder(input [3:0] x, input [3:0] y, input c_in,
                      output [3:0] sum, output c_out);
  assign {c_out, sum} = x + y + c_in;
endmodule
```

{c_out, sum}은 5비트 결과. MSB(bit 4)가 c_out, 나머지 4비트가 sum.

응용 예제: 4-bit Adder/Subtractor — 핵심 트릭!

```
module twos_adder(input [3:0] x, input [3:0] y, input c_in,
                  output [3:0] sum, output c_out);
  wire [3:0] t;
  assign t = y ^ {4{c_in}};           // c_in=0→t=y, c_in=1→t=~y
  assign {c_out, sum} = x + t + c_in; // c_in=1이면 x+~y+1 = x-y
endmodule
```

```
  c_in=0: t = y^0000 = y       → x + y + 0   = x + y    (덧셈)
  c_in=1: t = y^1111 = ~y      → x + ~y + 1  = x - y    (뺄셈, 2의 보
수)
```

이것이 하나의 가산기로 덧셈과 뺄셈을 모두 수행하는 핵심 트릭이다. {4{c_in}}은 replication 연산자로, c_in이 1이면 4'b1111, 0이면 4'b0000을 생성한다.

Conditional Operator (조건 연산자)

conditional_expression ? true_expression : false_expression. Mux나 tri-state 버퍼를 모델링하는 데 이상적.

```
assign out = selection ? in_1 : in_0;                          //
2:1 Mux
assign out = s1 ? (s0 ? i3 : i2) : (s0 ? i1 : i0);          // 4:1
Mux
```

condition이 x이면 양쪽 모두 평가하여 비트별로 비교. 같으면 그 값, 다르면 x.

Reduction Operators (축소 연산자)

벡터를 스칼라(1비트)로 축소. 단일 피연산자에 대해 비트별 연산을 누적.

```
// 패리티 생성기
assign ep = ^x;    // even parity: 모든 비트 XOR
assign op = ~ep;   // odd parity
```

```
// All-Zero / All-One 검출기
assign zero = ~(|x);   // 모든 비트 OR → 0이면 all-zero
assign one  = &x;      // 모든 비트 AND → 1이면 all-one
```

Logical Operators (논리 연산자)

!, &&, ||. Bitwise와의 차이: Logical은 전체 피연산자를 true/false로 평가하여 1비트 결과. Bitwise는 비트별 연산.

```
reg [7:0] c = 123, d = 0;
a = c || d;    // a = 1 (123은 true, 0은 false, true OR false = true)
b = c && d;    // b = 0 (true AND false = false)
```

Bitwise Operators (비트별 연산자)

~, &, |, ^, ~^. z는 bitwise에서 x로 취급된다. 짧은 피연산자는 zero-extend 후 연산.

x/z 비트 처리 진리표:

```
&  | 0  1  x  z        |  | 0  1  x  z
---+----------------   ---+----------------
0  | 0  0  0  0        0  | 0  1  x  x
1  | 0  1  x  x        1  | 1  1  1  1
x  | 0  x  x  x        x  | x  1  x  x
z  | 0  x  x  x        z  | x  1  x  x
```

Shift Operators (시프트 연산자)

논리 시프트(<<, >>): 빈 자리를 항상 0으로 채움. 산술 시프트(<<<, >>>): 좌시프트는 빈 자리 0(논리와 동일, 부호 비보존!). 우시프트는 부호 비트(sign bit)로 채움.

```
8비트 signed 값 = 10101010 (-86)

Logical >>1:     01010101            (0으로 채움)
Arithmetic >>>1: 11010101 (-43)      (부호비트 1로 채움)

Logical <<1:     01010100            (0으로 채움)
Arithmetic <<<1: 01010100 (84)       (0으로 채움, 부호 변경됨!)
```

함정: 산술 좌시프트는 논리 좌시프트와 동일하게 동작한다. 따라서 부호가 보존되지 않을 수 있다! 산술 우시프트만 부호 보존 효과가 있다.

Operator Precedence (연산자 우선순위)

높은 것부터 낮은 것 순서: ~ (NOT) > *, /, % > +, - > <<, >>, <<<, >>> > <, <=, >, >= > ==, !=, ===, !== > &, ~& > ^, ~^ > |, ~| > && > || > ?: (C 언어와 유사)